ОТЧЁТ

по лабораторной работе №10

«HTTP и REST API в Python»

Вариант 20. Интернет-провайдер, ресурс Tariff

1. Цель работы

Научиться описывать ресурс по правилам REST: методы HTTP, коды ответа, JSON, фильтрацию, сортировку, отдельный клиент и автоматические тесты API.

2. Постановка индивидуального задания

Вариант 20. Ресурс — тариф, специализированная операция — тарифы по скорости. Нужны получение списка и объекта, создание, полная и частичная замена, удаление, фильтр, сортировка и проверка входных данных.

Сервер написан на Flask. Клиент — отдельная консольная программа на httpx. Дополнительно требуются ключ в заголовке X-API-Key и постраничная выборка.

3. Краткие теоретические сведения

HTTP-запрос состоит из метода, пути, заголовков и необязательного тела. GET читает, POST создаёт, PUT заменяет, PATCH меняет часть полей, DELETE удаляет. Успешные коды в работе: 200, 201 и 204. Ошибки клиента: 400, 401 и 404.

REST рассматривает тариф как ресурс с адресом /tariffs и /tariffs/{id}. Сервер не запоминает меню клиента: каждое обращение содержит всё нужное, включая ключ. Слой маршрутов вызывает сервис, сервис проверяет данные, репозиторий хранит записи.

4. Описание программы

Тариф содержит название, скорость, цену, трафик, услуги, регион и статус. GET /tariffs принимает status, sort, page и size. GET /tariffs/by-speed — специализированный адрес варианта. Без заголовка X-API-Key ответ 401, адрес /health открыт для проверки, что сервер жив.

Некорректное название, нулевая скорость и отрицательная цена дают 400. Нет такого id — 404. Удаление успешного объекта возвращает пустое тело и код 204. Тесты покрывают список, отсутствие объекта, создание, ошибку

данных, PATCH, PUT, удаление, поиск по скорости, сортировку, отказ без ключа и фильтр статуса.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — app.py

```
"""
Лабораторная работа №10. Вариант 20.
REST API тарифов интернет-провайдера (Flask).
"""

from flask import Flask, jsonify, request

from repository import TariffRepository
from service import TariffService

API_KEY = "lab10-key"


def create_app(service=None):
    app = Flask(__name__)
    if service is None:
        service = TariffService(TariffRepository())
        service.create({"name": "Домашний 100", "speed": 100, "price": 500, "region": "Москва"})
        service.create({"name": "Домашний 300", "speed": 300, "price": 800, "region": "Казань"})
        service.create({
            "name": "Корпоративный 500", "speed": 500, "price": 2500,
            "region": "Москва", "services": ["белый IP"],
        })
    app.service = service

    @app.before_request
    def check_key():
        if request.path == "/health":
            return None
        if request.headers.get("X-API-Key") != API_KEY:
            return jsonify({"error": "нужен заголовок X-API-Key"}), 401

    @app.get("/health")
    def health():
        return jsonify({"status": "ok"})

    @app.get("/tariffs")
    def list_tariffs():
        status = request.args.get("status")
        sort = request.args.get("sort")
        min_speed = request.args.get("min_speed")
        page = int(request.args.get("page", 1))
        size = int(request.args.get("size", 20))
        items, total = service.list_tariffs(status, sort, min_speed, page, size)
        return jsonify({"items": items, "total": total, "page": page, "size": size})

    @app.get("/tariffs/by-speed")
    def by_speed():
        min_speed = request.args.get("min_speed", type=int)
        if min_speed is None:
            return jsonify({"error": "укажите min_speed"}), 400
        return jsonify(service.by_speed(min_speed))

    @app.get("/tariffs/<int:item_id>")
    def get_tariff(item_id):
        item = service.repository.get(item_id)
        if item is None:
            return jsonify({"error": "тариф не найден"}), 404
        return jsonify(item)
```

```
    @app.post("/tariffs")
    def create_tariff():
        data = request.get_json(silent=True)
        if not isinstance(data, dict):
            return jsonify({"error": "ожидался JSON"}), 400
        try:
            item = service.create(data)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        return jsonify(item), 201

    @app.put("/tariffs/<int:item_id>")
    def replace_tariff(item_id):
        data = request.get_json(silent=True) or {}
        if service.repository.get(item_id) is None:
            return jsonify({"error": "тариф не найден"}), 404
        try:
            service.validate(data, partial=False)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        payload = {
            "name": data["name"].strip(),
            "speed": int(data["speed"]),
            "price": float(data["price"]),
            "traffic": data.get("traffic", "безлимит"),
            "services": data.get("services") or [],
            "region": data.get("region", "Москва"),
            "status": data.get("status", "активен"),
        }
        return jsonify(service.repository.update(item_id, payload, partial=False))

    @app.patch("/tariffs/<int:item_id>")
    def patch_tariff(item_id):
        data = request.get_json(silent=True) or {}
        try:
            service.validate(data, partial=True)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        if "speed" in data:
            data["speed"] = int(data["speed"])
        if "price" in data:
            data["price"] = float(data["price"])
        item = service.repository.update(item_id, data, partial=True)
        if item is None:
            return jsonify({"error": "тариф не найден"}), 404
        return jsonify(item)

    @app.delete("/tariffs/<int:item_id>")
    def delete_tariff(item_id):
        if not service.repository.delete(item_id):
            return jsonify({"error": "тариф не найден"}), 404
        return "", 204

    @app.errorhandler(405)
    def method_not_allowed(_exc):
        return jsonify({"error": "метод не поддерживается"}), 405

    return app


if __name__ == "__main__":
    create_app().run(host="127.0.0.1", port=5010, debug=False)
```

Листинг — client.py

```
"""Консольный HTTP-клиент лабораторной работы №10."""

import json
import sys

import httpx

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
```

```
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")

BASE = "http://127.0.0.1:5010"
HEADERS = {"X-API-Key": "lab10-key"}



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def show(response: httpx.Response):
    print(f"HTTP {response.status_code}")
    if response.content:
        try:
            print(json.dumps(response.json(), ensure_ascii=False, indent=2))
        except json.JSONDecodeError:
            print(response.text)
    else:
        print("(пустое тело)")



def print_menu():
    print()
    print("===== HTTP-клиент тарифов =====")
    print("1. Получить список")
    print("2. Получить объект")
    print("3. Создать")
    print("4. Изменить (PUT)")
    print("5. Частично изменить (PATCH)")
    print("6. Удалить")
    print("7. Фильтрация по статусу")
    print("8. Сортировка по цене")
    print("9. Тарифы по скорости")
    print("0. Выход")
    print()



def main():
    client = httpx.Client(base_url=BASE, headers=HEADERS, timeout=5)
    try:
        health = httpx.get(BASE + "/health", timeout=3)
        print("Сервер:", health.json())
    except httpx.HTTPError as exc:
        print("Сервер не запущен:", exc)
        print("Запустите: python app.py")
        return

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                show(client.get("/tariffs"))
            elif choice == "2":
                show(client.get(f"/tariffs/{read_line('ID: ')}"))
            elif choice == "3":
                payload = {
                    "name": read_line("Название: "),
                    "speed": int(read_line("Скорость: ")),
                    "price": float(read_line("Цена: ").replace(",", ".")),
                    "region": read_line("Регион: "),
                    "traffic": "безлимит",
                }
                show(client.post("/tariffs", json=payload))
            elif choice == "4":
```

```
                item_id = read_line("ID: ")
                payload = {
                    "name": read_line("Название: "),
                    "speed": int(read_line("Скорость: ")),
                    "price": float(read_line("Цена: ").replace(",", ".")),
                    "region": read_line("Регион: "),
                    "status": "активен",
                }
                show(client.put(f"/tariffs/{item_id}", json=payload))
            elif choice == "5":
                item_id = read_line("ID: ")
                price = float(read_line("Новая цена: ").replace(",", "."))
                show(client.patch(f"/tariffs/{item_id}", json={"price": price}))
            elif choice == "6":
                show(client.delete(f"/tariffs/{read_line('ID: ')}"))
            elif choice == "7":
                show(client.get("/tariffs", params={"status": "активен"}))
            elif choice == "8":
                show(client.get("/tariffs", params={"sort": "price"}))
            elif choice == "9":
                speed = read_line("Минимальная скорость: ")
                show(client.get("/tariffs/by-speed", params={"min_speed": speed}))
            elif choice == "0":
                print("Работа клиента завершена.")
                break
            else:
                print("Ошибка: выберите пункт от 0 до 9.")
        except (httpx.HTTPError, ValueError) as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            break
    client.close()


if __name__ == "__main__":
    main()
```

Листинг — models.py

```
"""Тариф как ресурс REST API."""

from dataclasses import dataclass, field, asdict


@dataclass
class Tariff:
    id: int
    name: str
    speed: int
    price: float
    traffic: str
    services: list[str] = field(default_factory=list)
    region: str = "Москва"
    status: str = "активен"

    def to_dict(self):
        return asdict(self)
```

Листинг — repository.py

```
"""Хранилище тарифов в памяти. HTTP-слой его не создаёт сам."""


class TariffRepository:
    def __init__(self):
        self._items = {}
        self._next = 1

    def add(self, data: dict):
        item_id = self._next
        self._next += 1
        data = dict(data)
        data["id"] = item_id
        self._items[item_id] = data
        return data
```

```
    def all(self):
        return list(self._items.values())

    def get(self, item_id: int):
        return self._items.get(item_id)

    def update(self, item_id: int, data: dict, partial: bool):
        current = self._items.get(item_id)
        if current is None:
            return None
        if partial:
            current.update({key: value for key, value in data.items() if key != "id"})
        else:
            current.clear()
            current.update(data)
            current["id"] = item_id
        return current

    def delete(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None
```

Листинг — service.py

```
"""Бизнес-правила тарифа, отдельно от маршрутов Flask."""

STATUSES = {"активен", "архив"}



class TariffService:
    def __init__(self, repository):
        self.repository = repository

    def validate(self, data: dict, partial: bool = False):
        required = ["name", "speed", "price"]
        if not partial:
            missing = [key for key in required if key not in data]
            if missing:
                raise ValueError("не заполнены поля: " + ", ".join(missing))
        if "speed" in data and int(data["speed"]) <= 0:
            raise ValueError("скорость должна быть больше нуля")
        if "price" in data and float(data["price"]) < 0:
            raise ValueError("цена не может быть отрицательной")
        if "status" in data and data["status"] not in STATUSES:
            raise ValueError("статус: активен или архив")
        if "name" in data and not str(data["name"]).strip():
            raise ValueError("название не должно быть пустым")

    def create(self, data: dict):
        self.validate(data)
        payload = {
            "name": data["name"].strip(),
            "speed": int(data["speed"]),
            "price": float(data["price"]),
            "traffic": data.get("traffic", "безлимит"),
            "services": data.get("services") or [],
            "region": data.get("region", "Москва"),
            "status": data.get("status", "активен"),
        }
        return self.repository.add(payload)

    def list_tariffs(self, status=None, sort=None, min_speed=None, page=1, size=20):
        items = self.repository.all()
        if status:
            items = [item for item in items if item["status"] == status]
        if min_speed is not None:
            items = [item for item in items if item["speed"] >= int(min_speed)]
        if sort == "price":
            items = sorted(items, key=lambda item: item["price"])
        elif sort == "speed":
            items = sorted(items, key=lambda item: item["speed"], reverse=True)
        start = (page - 1) * size
        return items[start:start + size], len(items)
```

```
    def by_speed(self, min_speed: int):
        return [item for item in self.repository.all() if item["speed"] >= min_speed]
```

Листинг — tests\test_api.py

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app import API_KEY, create_app
from repository import TariffRepository
from service import TariffService



def client():
    service = TariffService(TariffRepository())
    service.create({"name": "Домашний 100", "speed": 100, "price": 500})
    service.create({"name": "Быстрый 300", "speed": 300, "price": 900})
    application = create_app(service)
    application.testing = True
    test_client = application.test_client()
    test_client.environ_base = {}
    return test_client



HEADERS = {"X-API-Key": API_KEY}



def test_list_ok():
    response = client().get("/tariffs", headers=HEADERS)
    assert response.status_code == 200
    assert response.get_json()["total"] == 2



def test_get_missing():
    response = client().get("/tariffs/99", headers=HEADERS)
    assert response.status_code == 404



def test_create():
    response = client().post("/tariffs", headers=HEADERS, json={
        "name": "Ночной", "speed": 50, "price": 300, "region": "Тула",
    })
    assert response.status_code == 201
    assert response.get_json()["id"] == 3



def test_create_invalid():
    response = client().post("/tariffs", headers=HEADERS, json={"name": "", "speed": 10, "price": 1})
    assert response.status_code == 400



def test_patch_price():
    response = client().patch("/tariffs/1", headers=HEADERS, json={"price": 450})
    assert response.status_code == 200
    assert response.get_json()["price"] == 450



def test_put_replace():
    response = client().put("/tariffs/1", headers=HEADERS, json={
        "name": "Обновлённый", "speed": 200, "price": 700, "region": "Москва",
    })
    assert response.status_code == 200
    assert response.get_json()["name"] == "Обновлённый"



def test_delete():
    api = client()
    response = api.delete("/tariffs/1", headers=HEADERS)
```

```
    assert response.status_code == 204
    assert api.get("/tariffs/1", headers=HEADERS).status_code == 404


def test_by_speed():
    response = client().get("/tariffs/by-speed?min_speed=200", headers=HEADERS)
    assert response.status_code == 200
    assert len(response.get_json()) == 1


def test_sort_price():
    response = client().get("/tariffs?sort=price", headers=HEADERS)
    prices = [item["price"] for item in response.get_json()["items"]]
    assert prices == sorted(prices)


def test_unauthorized():
    response = client().get("/tariffs")
    assert response.status_code == 401


def test_filter_status():
    api = client()
    api.patch("/tariffs/2", headers=HEADERS, json={"status": "архив"})
    response = api.get("/tariffs?status=архив", headers=HEADERS)
    assert response.get_json()["total"] == 1
```

6. Примеры работы программы

Снимки клиента показывают список и поиск по скорости, создание тарифа, сортировку и фильтр, а также прогон тестов API.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python10> python client.py

Сервер: {'status': 'ok'}

===== HTTP-клиент тарифов =====
1. Получить список
2. Получить объект
3. Создать
4. Изменить (PUT)
5. Частично изменить (PATCH)
6. Удалить
7. Фильтрация по статусу
8. Сортировка по цене
9. Тарифы по скорости
0. Выход

Выберите действие: 1
HTTP 200
{
  "items": [
    {
      "id": 1,
      "name": "Домашний 100",
      "price": 500.0,
      "region": "Москва",
      "services": [],
      "speed": 100,
      "status": "активен",
      "traffic": "безлимит"
    },
    {
      "id": 2,
      "name": "Домашний 300",
      "price": 800.0,
      "region": "Казань",
      "services": [],
      "speed": 300,
      "status": "активен",
      "traffic": "безлимит"
    },
    {
      "id": 3,
      "name": "Корпоративный 500",
      "price": 2500.0,
      "region": "Москва",
      "services": [
        "белый IP"
      ],
      "speed": 500,
      "status": "активен",
```

Рисунок 1 – Список тарифов и поиск по минимальной скорости

```
C:\\Users\\Shmurdyakich\\Desktop\\pythonALL\\python10> python -m pytest -q

...........                                                              [100%]
11 passed in 0.13s
```

Рисунок 2 – Автоматические тесты REST API

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python10> python client.py

Сервер: {'status': 'ok'}

===== HTTP-клиент тарифов =====
1. Получить список
2. Получить объект
3. Создать
4. Изменить (PUT)
5. Частично изменить (PATCH)
6. Удалить
7. Фильтрация по статусу
8. Сортировка по цене
9. Тарифы по скорости
0. Выход

Выберите действие: 3
Название: Ночной
Скорость: 100
Цена: 300
Регион: Тула
HTTP 201
{
  "id": 5,
  "name": "Ночной",
  "price": 300.0,
  "region": "Тула",
  "services": [],
  "speed": 100,
  "status": "активен",
  "traffic": "безлимит"
}

===== HTTP-клиент тарифов =====
1. Получить список
2. Получить объект
3. Создать
4. Изменить (PUT)
5. Частично изменить (PATCH)
6. Удалить
7. Фильтрация по статусу
8. Сортировка по цене
9. Тарифы по скорости
0. Выход

Выберите действие: 8
HTTP 200
{
  "items": [
    {
```

Рисунок 3 – Создание тарифа, сортировка по цене и фильтр по статусу

7. Ответы на контрольные вопросы

1. Какой метод создаёт ресурс?

POST. При успехе сервер отвечает 201 Created и телом нового тарифа.

2. Чем PUT отличается от PATCH?

PUT передаёт новое полное состояние. PATCH меняет только присланные поля, остальные остаются.

3. Почему DELETE отвечает 204?

Объект удалён, и возвращать тело нечего. Код 204 как раз означает успех без содержимого.

4. Что такое 404 в этой работе?

Тарифа с указанным id нет. Это ошибка адреса ресурса, а не поломка сервера.

5. Зачем отдельный клиент?

Он показывает, что API можно вызвать из другой программы, а не только из браузера или тестов.

6. Почему ключ передаётся заголовком?

Это данные о правах запроса, а не поле тарифа. Без заголовка список не отдаётся.

7. Где проходит граница слоёв?

Flask принимает HTTP, сервис проверяет правила, репозиторий хранит словари. Маршрут не считает статистику сам.

8. Вывод

В лабораторной работе №10 тарифы варианта 20 опубликованы как REST-ресурс. Реализованы CRUD, поиск по скорости, фильтр, сортировка, коды ошибок и ключ доступа. Консольный клиент и тесты подтверждают контракт API.